

Segment Anything in High Quality (HQ-SAM)

이미지 처리팀

강인하, 김병현, 김준철, 류채은, 안종식, 조경진, 최승준

Content

- SAM (Segment Anything)
 - Demo
 - Motivation
 - Task
 - Model
 - Data
 - Experiment
- HQ-SAM
 - Motivation
 - Related Works
 - Method
 - Experiment

SAM-Demo

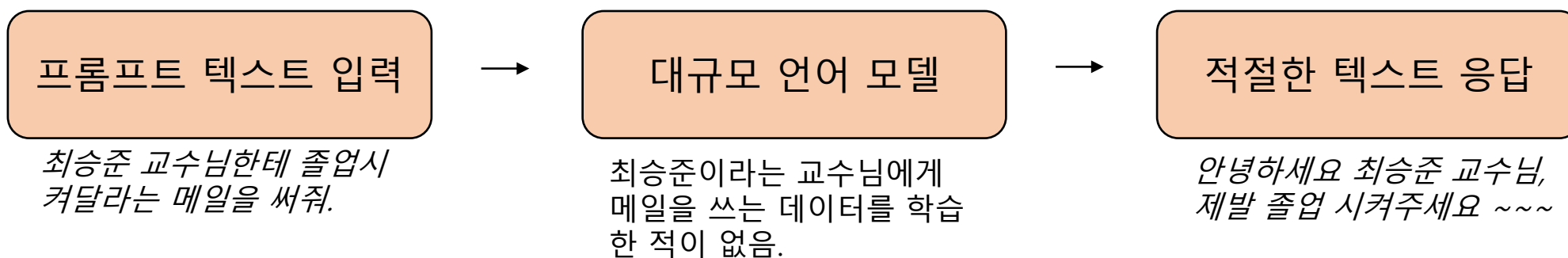


SAM (Segment Anything)

- 주어진 Prompt (점, 박스, Text)에 대해 어떤 대상이든 Segmentation Mask를 생성해주는 모델.

SAM-Motivation

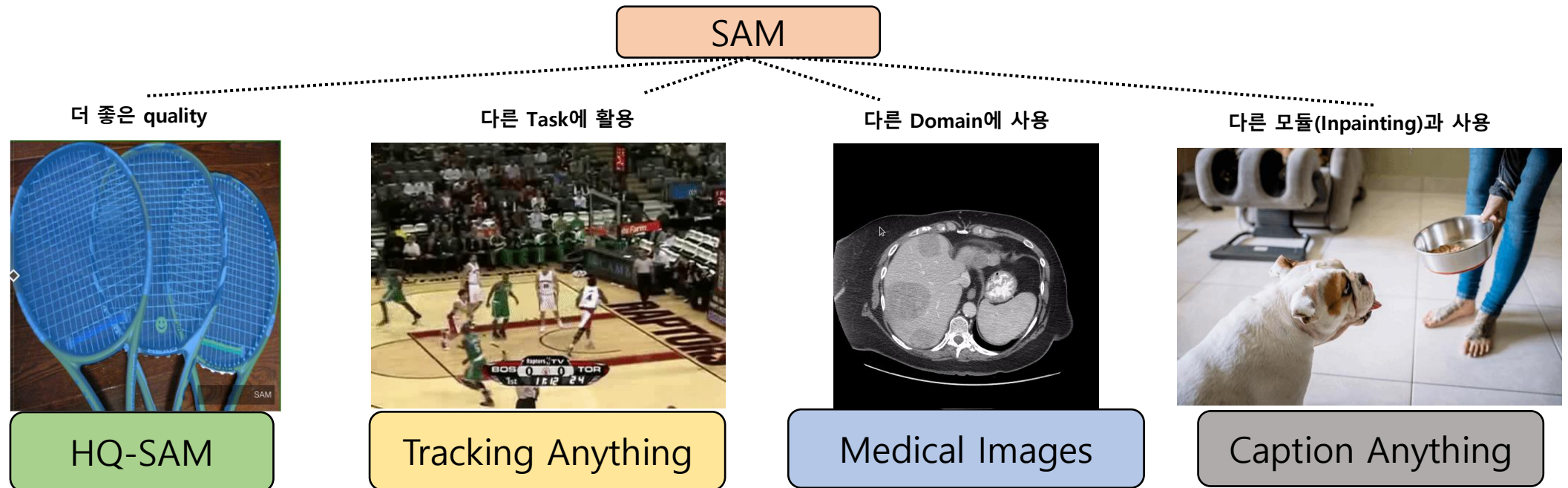
- NLP의 발전
 - LLM (Large Language model) 웹에서의 대규모 데이터 세트를 비지도 (Unsupervised) 사전학습
-> **Foundation model**
 - **Prompt Engineering**을 통해 다양한 기능(Downstream Task)들이 구현됨.
 - 웹상에서의 거대한 텍스트 말뭉치 데이터 -> finetuning 모델과 비교해 zero & few-shot 성능을 향상시킴



- Computer vision (Segmentation task)의 Foundation model을 만들 수 있을까? -> **Segment Anything (SAM)**
 - Text-Image 쌍 데이터 학습 (CLIP) – 웹상에서 이미지 캡션 데이터
 - 하지만, Object Detection. Segmentation 등에 쓰이는 거대한 데이터는 찾기 어려움.

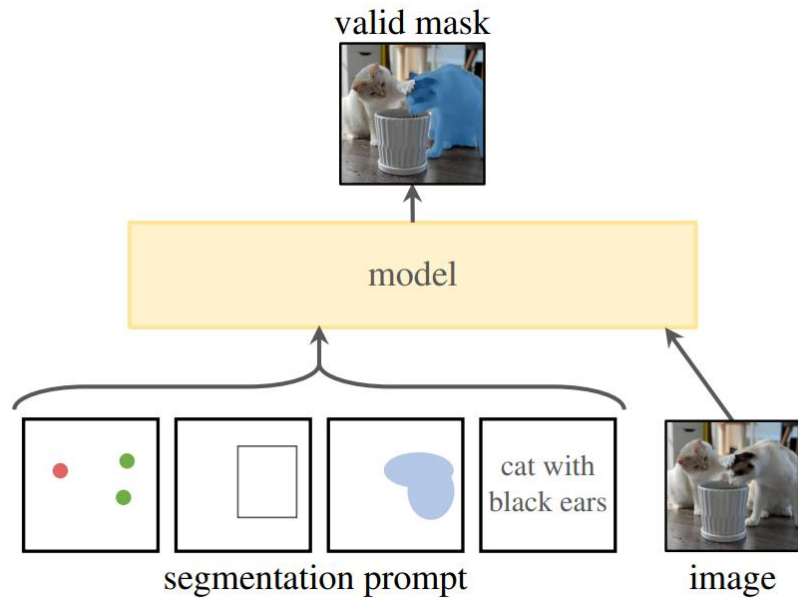
SAM Motivation

- SAM 목표: 이미지 segmentation에 적용할 수 있는 **foundation model**을 만들자!
- 구현하기 위한 세 가지 구성 요소: Task, Model, and Data
 - zero-shot generalization이 가능한 task? -> Promptable segmentation task
 - Task 에 대한 Model architecture는 어떻게? -> Flexible Prompting
 - 어떤 Data를 모아야 될까? -> 다양한 출처에서 가져온 Large-scale of image segmentation data



SAM-Task

- Promptable Segmentation Task
 - 주어진 어떠한 Prompts라도 valid한 segmentation mask를 출력하는 것.



이미지와 세그먼트할 대상을 지정
(포인트, 바운딩 박스, 마스크, 텍스트)



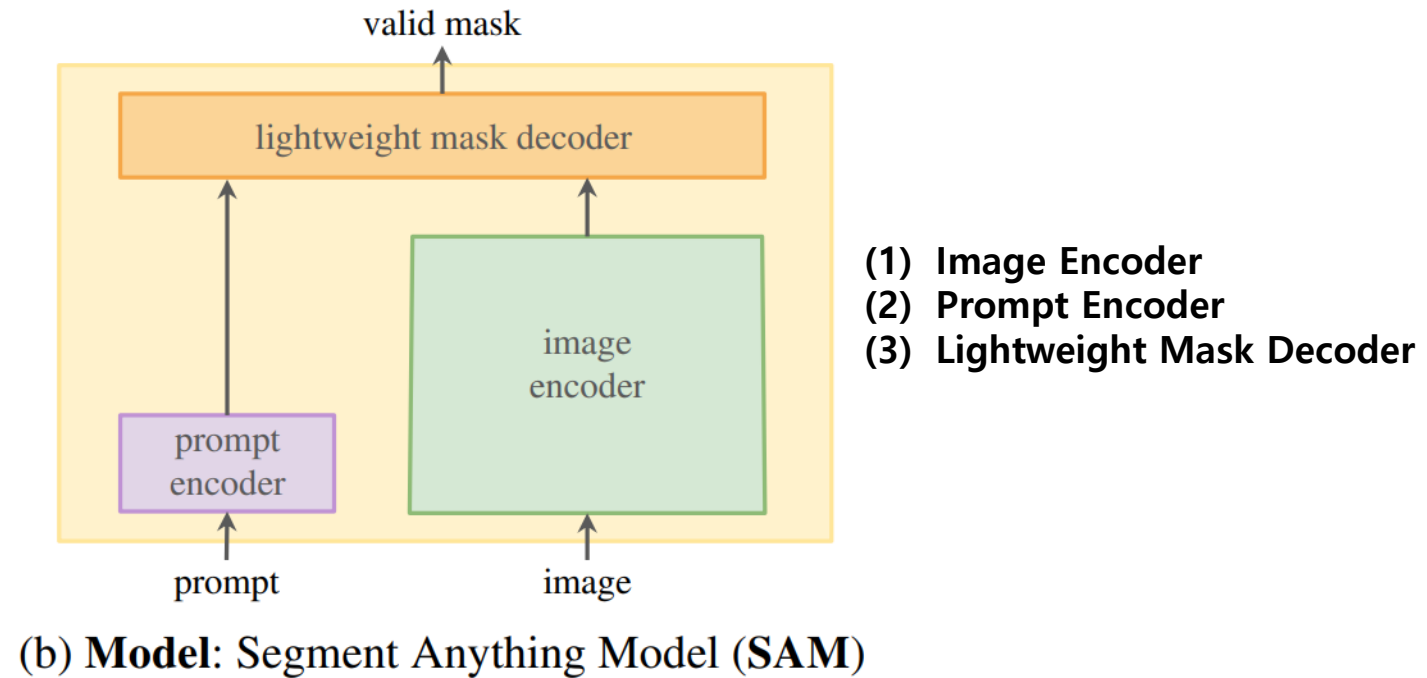
프롬프트가 모호하고 여러 개체를 참조할 수 있는 경우, 출력은 해당 객체 중 적어도 하나에 대한 합리적인 마스크로 나오게 됨

SAM-Model

- 모델 요구사항

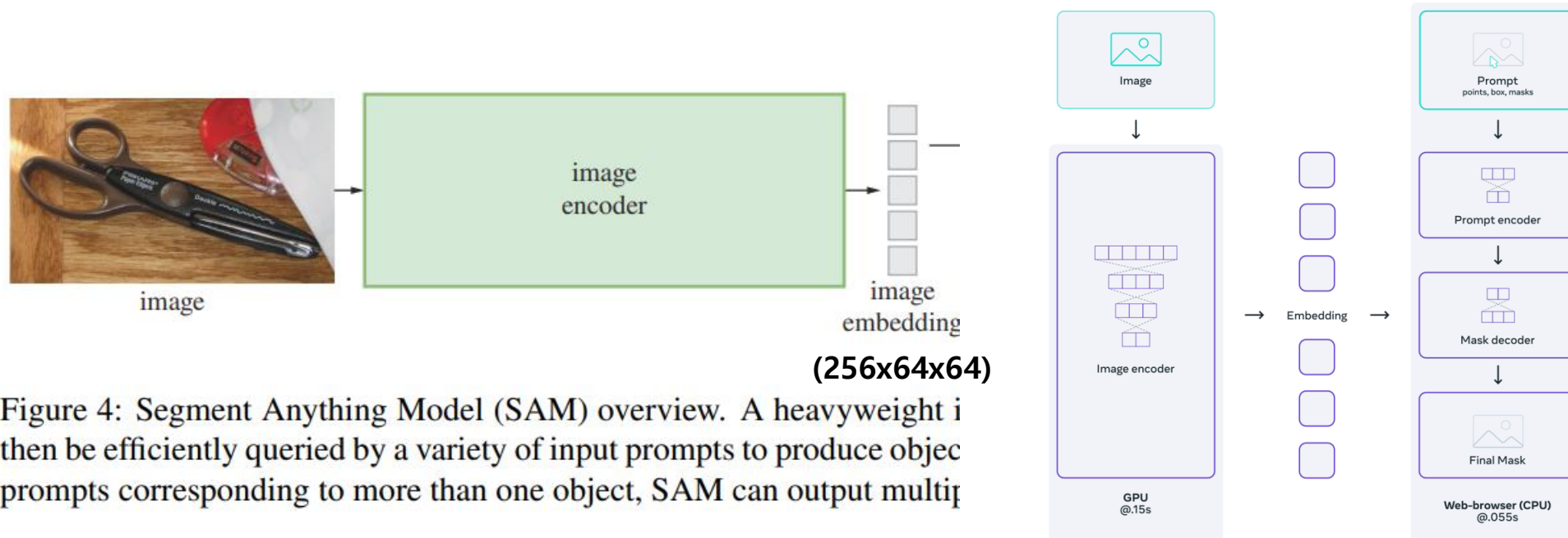
- 다양한 프롬프트를 지원할 것
- 프롬프트로 Interactive하게 사용하기 위해 mask들을 실시간으로 분할해서 계산해야 한다.
- 이미지와 Prompt의 모호성을 인식할 것

Prompt(text): 옷



SAM-Model

- 모델 아키텍처

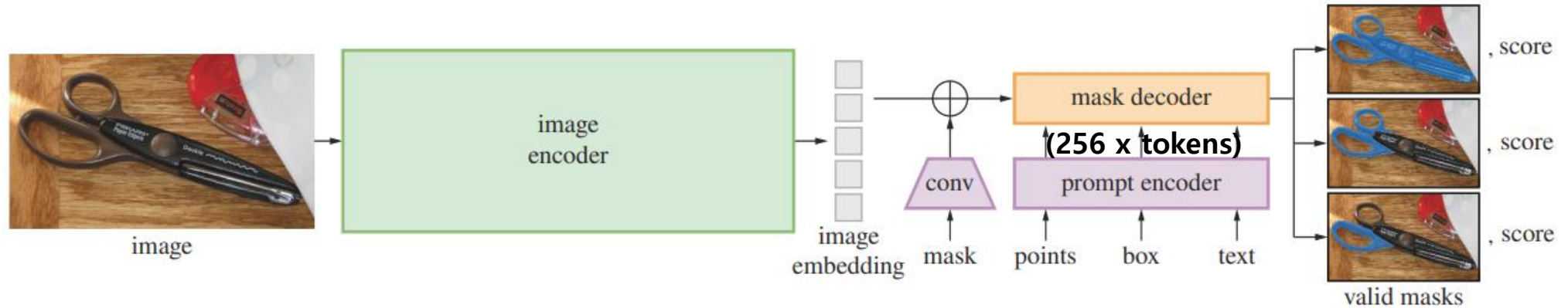


- Image Encoder

- **Masked Autoencoder (MAE)** pre-trained Vision Transformer (ViT): ViT-H/16 사용
- Prompt 마다 Image encoder가 실행되지 않고, 이미지마다 한번만 계산해서 Image embedding (256x64x64) 계산
- Image encoding의 경우 GPU 를 사용하여 0.15초 걸림.
- The prompt encoder and mask decoder를 web browser에서 CPU를이용해 mask를 50ms 안에 예측한다.

SAM-Model

- 모델 아키텍처

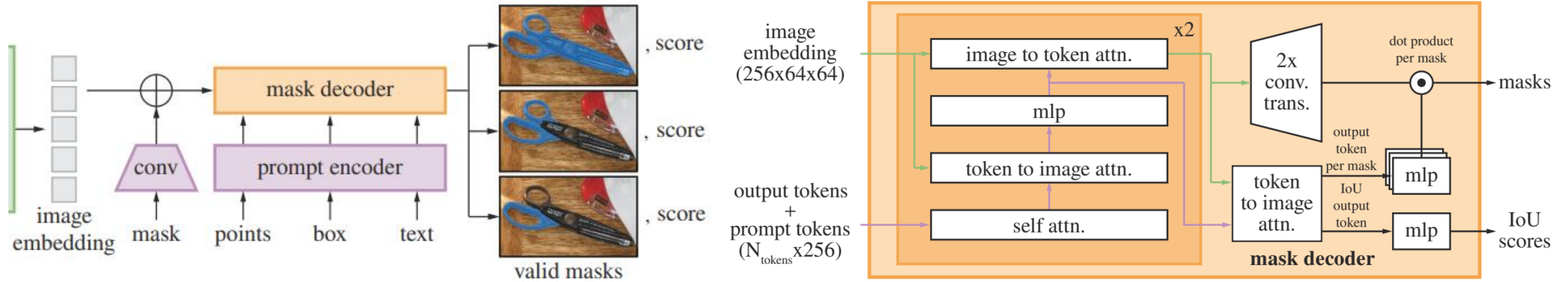


- Prompt Encoder**

- Sparse** prompts (points, boxes, text): 256 차원의 벡터로 Embedding 됨.
 - Points: 점의 위치를 positional encoding + foreground/background embedding (learned) (256 x tokens)
 - Boxes:
 - 왼쪽 상단 위치를 positional encoding + "top-left corner" 대표되는 learned embedding representing the same structure but using a learned embedding indicating "bottom-right corner"
 - Text: text encoder from CLIP
- Dense** prompts (masks): convolutions and summed element-wise with the image embedding

SAM-Model

- 모델 아키텍처

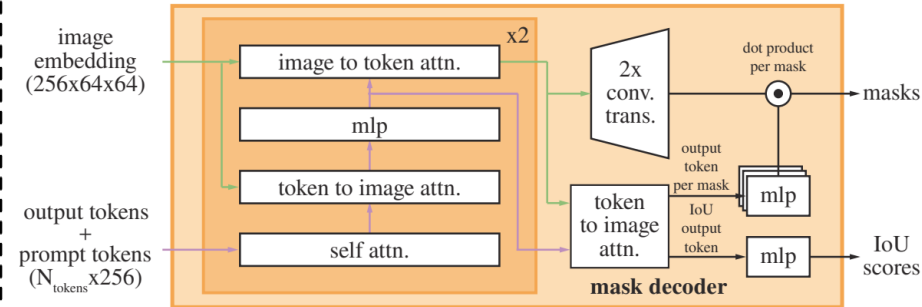
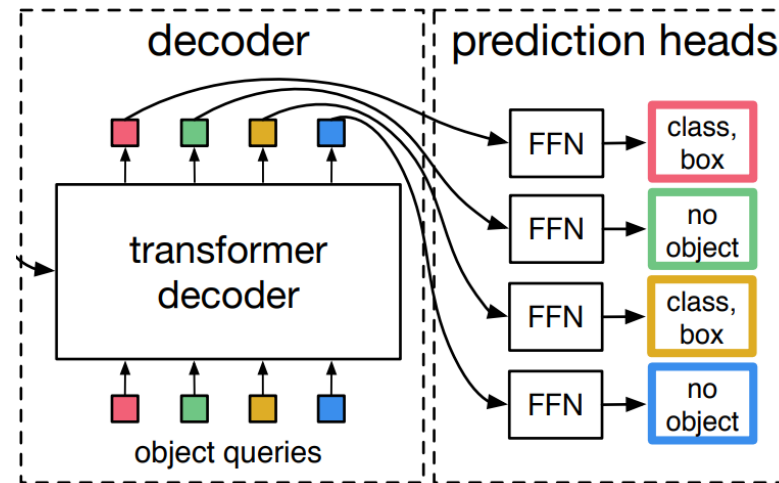
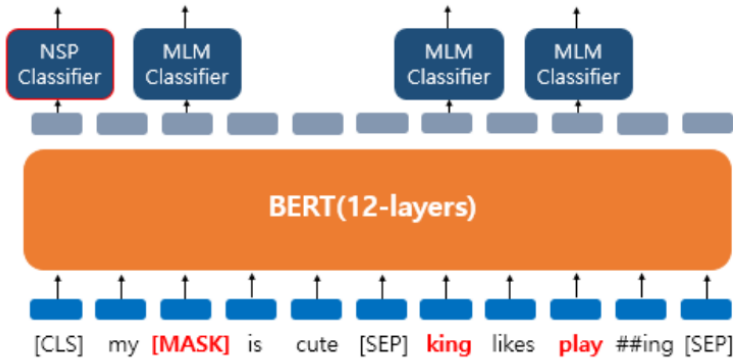


- Mask Decoder (Lightweight)**

- (1) self-attention on the tokens
- (2) cross-attention from tokens to the image embedding (**query: tokens, key,value: image**)
-> **token에 따른 주어진 image 가중치**
- (3) point-wise MLP updates each token
-> **pixel 별 channel끼리 linear layer**
- (4) cross-attention from the image embedding to tokens (**query: image, key,value: tokens**)
-> **image에 따른 주어진 token 가중치**
- (5) token to image cross attention 한번 더 해서 Output token을 뽑아낸다.
- (6) Attention을 통과한 image embedding을 convtranspose 이용해서 spatial size를 4배씩 늘린다.
- (7) Output token 중 일부는 image embedding과 같이 dot product해서 mask를, 나머지 하나는 IoU Score를 추정한다.

SAM-Model

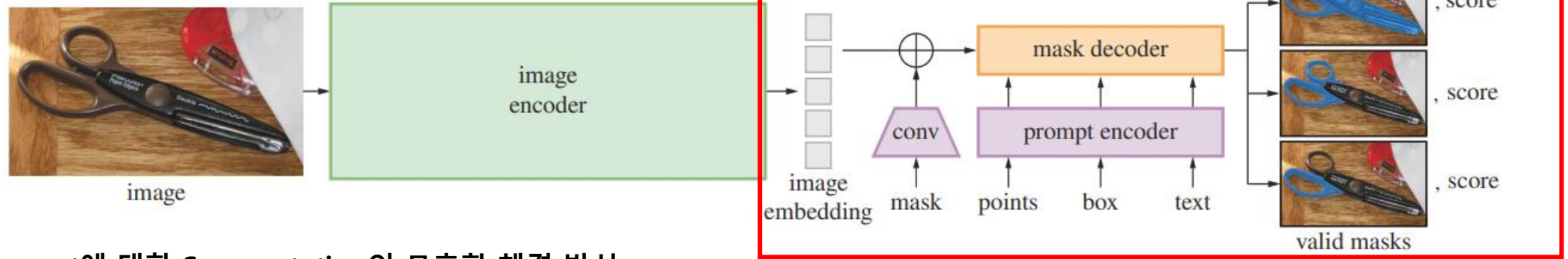
- Output 토큰(token)?
- SAM 논문에서 Decoder Input으로 attention을 통해 image와 prompt에 대한 정보를 갖는다.
(원래는 의미가 없는 값)
- BERT ViT [CLS] token, DETR object query, SAM Output token(Token) 개념이 모두 비슷.



- 문장 전체를 대표하는 임베딩 결과를 [CLS]로 보고 classification task를 할 때 사용
- Object queries가 decoder를 지나 class와 bounding box가 될 수 있도록 학습.
- Output token이 IOU score와 mask를 예측할 수 있도록 학습.

SAM-Model

- 모델 아키텍처

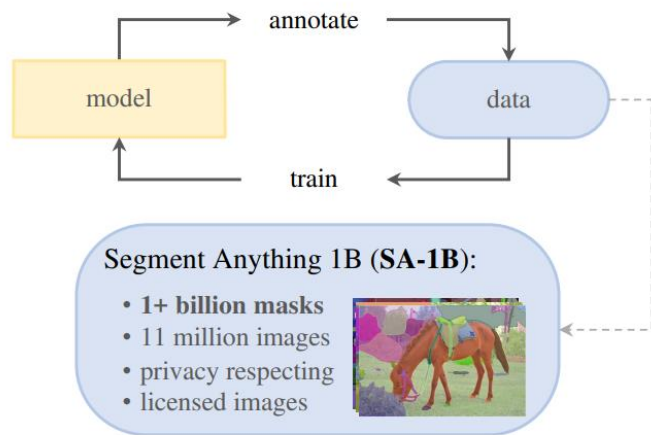


- Prompt에 대한 Segmentation의 모호함 해결 방식

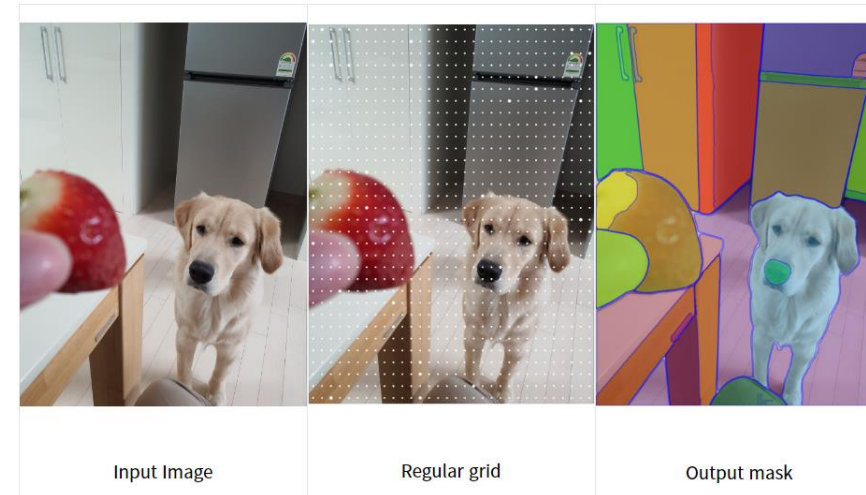
- 3개의 mask를 예측함: 전체, 파트(part), 서브파트 (서브파트)
- 각각의 예측한 마스크와 ground truth 사이의 loss를 계산하고 가장 낮은 loss 만 backpropagation함.
- 예측한 마스크와 실제 ground truth 사이의 IOU(score)를 예측할 수 있는 작은 head를 더함.
- 마스크 예측 Loss: Focal Loss + dice loss (20:1) 사용
- IOU 예측 head loss: MSE Loss 사용

SAM-Data

- 1.1B mask dataset, SA-1B 약 11 억개의 이미지를 학습시킴: 99.1%의 마스크를 자동으로 생성



(c) **Data:** data engine (top) & dataset (bottom)



<Fully automatic stage 예시>

- 데이터 엔진(Labeling 방식)은 총 3가지 스테이지가 있다.

1. Model-assisted manual annotation stage: 작업자가 점을 찍으면 SAM 모델이 mask를 만들어줌.
2. Semi-automatic stage: 특정 object 집합에 대해 SAM이 알아서 mask를 만들면 동시에 작업자가 다른 object에 대해 마스크를 만든다.
3. Fully automatic stage: 이미지에 Grid Point를 찍어서 알아서 Masking 하게 하는것

SAM-Experiment

- Zero-Shot Transfer Experiments

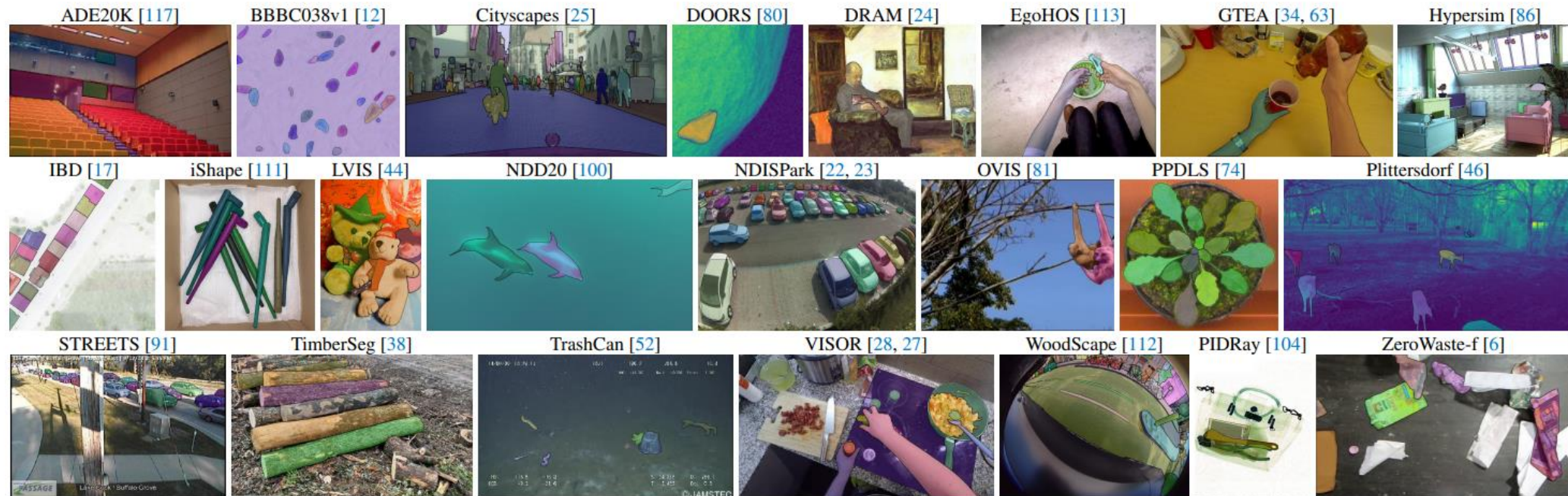
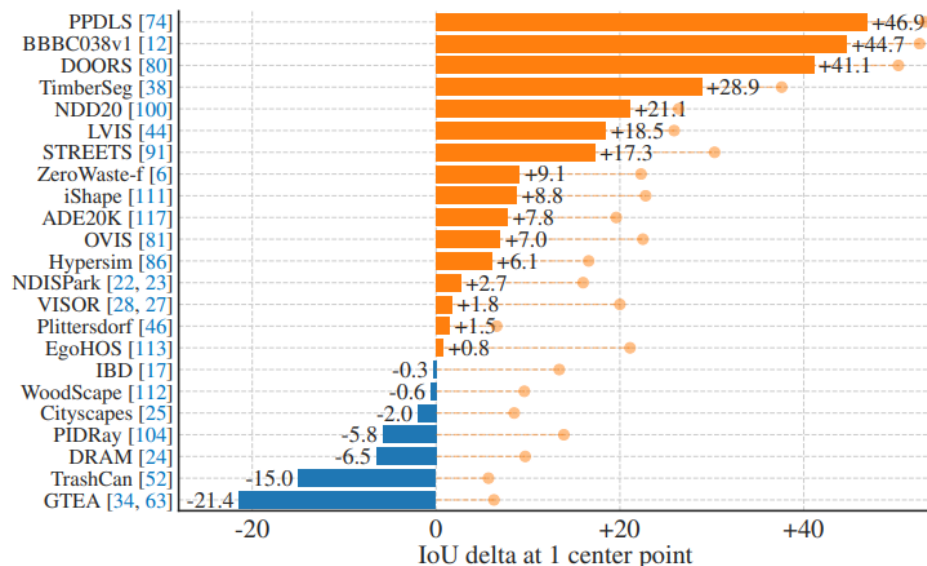


Figure 8: Samples from the 23 diverse segmentation datasets used to evaluate SAM's zero-shot transfer capabilities.

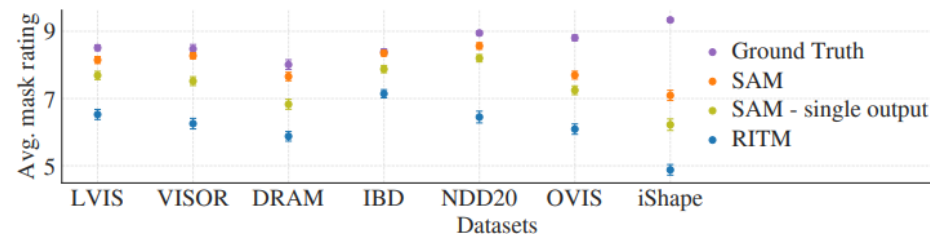
- 평가 데이터셋은 희귀하고 생소한 이미지들 (Novel Image distributions)을 갖고 있고, 학습데이터(SA-1B)에 포함되어 있지 않음.

SAM-Experiment

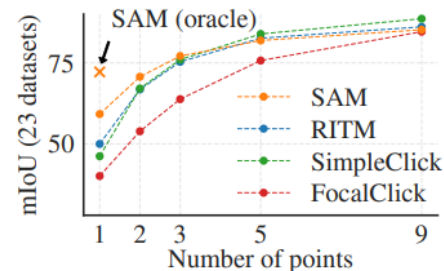
- Zero-Shot Single Point Valid Mask Evaluation Task



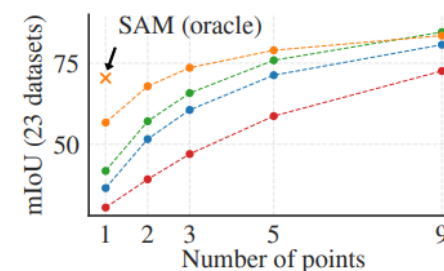
(a) SAM vs. RITM [92] on 23 datasets



(b) Mask quality ratings by human annotators



(c) Center points (default)



(d) Random points

- 점 하나(Prompt)에 대해 segmentation한 결과
- SAM이 RITM에 비해 23개의 결과데이터셋 중에서 16개의 데이터셋에서 높은 성능을 보여줌.
- SAM (oracle)은 SAM의 3개 마스크에 대해서 ground-truth와 비교해서 가장 관련 있는 것으로 비교한 결과임.

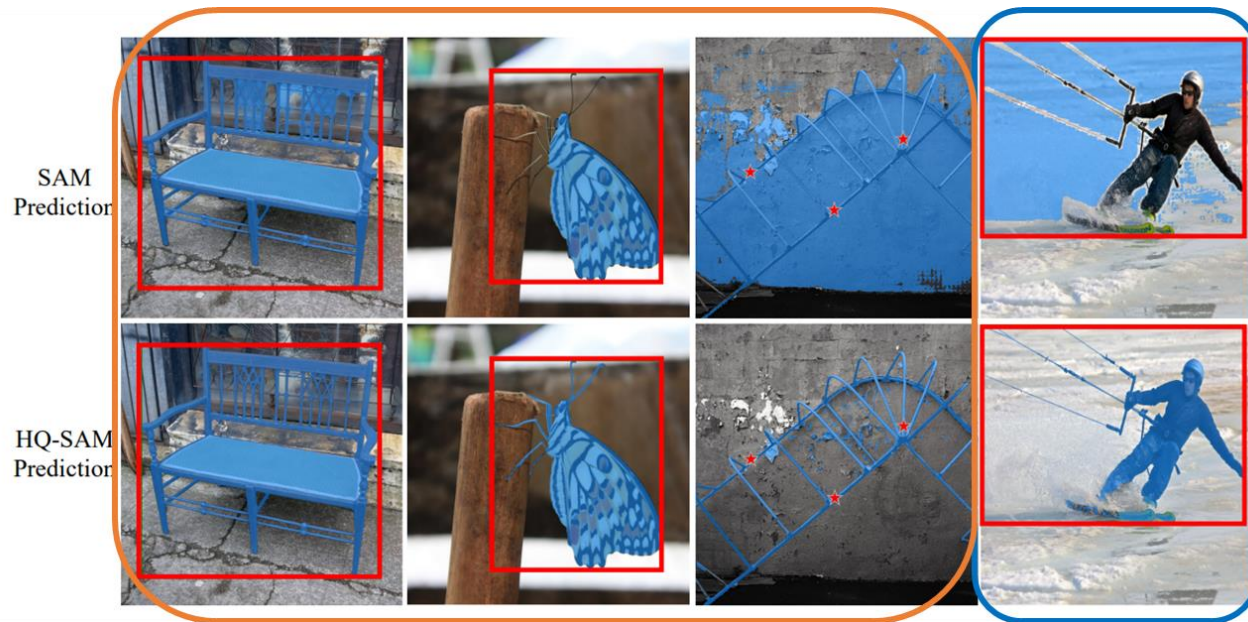
- Human annotator가 mask quality 점수를 매겼을 때 비교
- SAM Mask quality가 꽤 좋음.**
- SAM (oracle)은 SAM의 3개 마스크에 대해서 ground-truth와 비교해서 가장 관련 있는 것으로 비교한 결과임.

Question?

Question & Answer

HQ-SAM Motivation

- SAM의 문제
- **Coarse mask boundaries:** 얇은 선 형태의 물체 segmentation을 잘 못하는 것
- **Incorrect Prediction:** 얇은 구조물을 모델에서 잘못 해석해서, 이상한 mask를 만들어냄.



- SAM이라는 Foundation Model을 **Fine-tuning**으로 직접 바꾸지 않고 어떻게 하면 효율적으로 성능을 향상시킬 수 있을까?
- 오직 SAM 전체 parameter의 0.5%정도만 추가했음에도 높은 성능 달성
- 얇은 물체 segmentation 못하는 기존의 SAM 문제 해결

Related Works

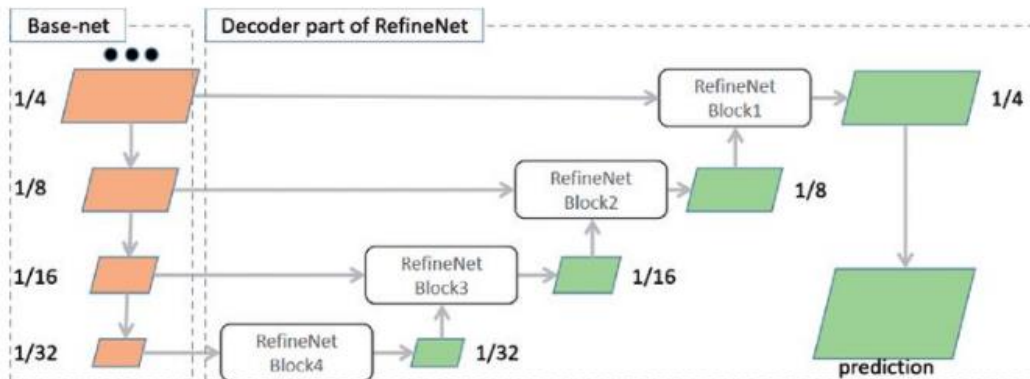
- High-quality Segmentation

- Input Image와 동일한 해상도의 Segmentation을 얻는 과정에서, Feature Map의 해상도를 유지하면 좋겠으나 Memory 문제로 해상도 유지불가
- 논문에서는 High-quality segmentation을 위한 접근으로 크게 두가지를 소개
- 1. Prediction Mask Quality 향상, 2. Post Processing(mask prediction 후 refinement)

- Prediction Mask quality 향상

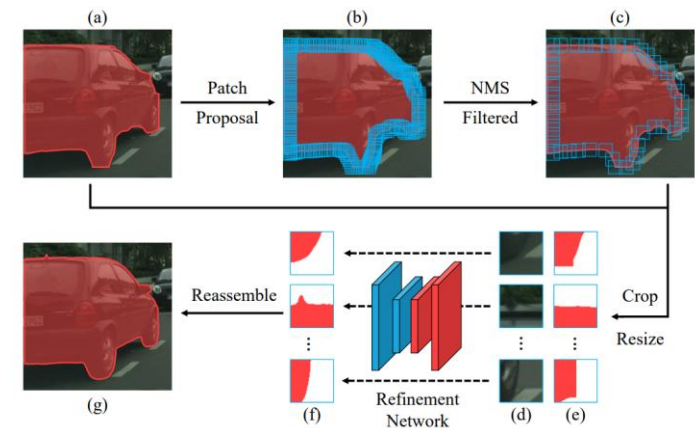
RefineNet: Multi-Path Refinement Networks for High-Resolution Semantic Segmentation

- Semantic Segmentation에서 많이 사용되는 Unet과 유사한 구조 (Encoder-Decoder)에
- Refinement Block을 붙여서 Multi Scale Feature Map을 이용한 Refinement Network를 만듦.
- Path를 여러 개 붙여 여러 개의 Input을 넣을 수 있도록 함.



Mask transfiner for high-quality instance segmentation

- Base detector로 거친 마스크(Coarse Mask)를 만든 뒤 이미지 scale별로 일관되지 않은 부분의 boundary영역을 파악해 refine함.
- Refinement Network로 HRNetV2 사용
- HRNetV2는 Semantic Segmentation Network
- Proposal Area에서 Binary Mask Quality만 Refinement하기 때문에 1 class Segmentation을 진행.



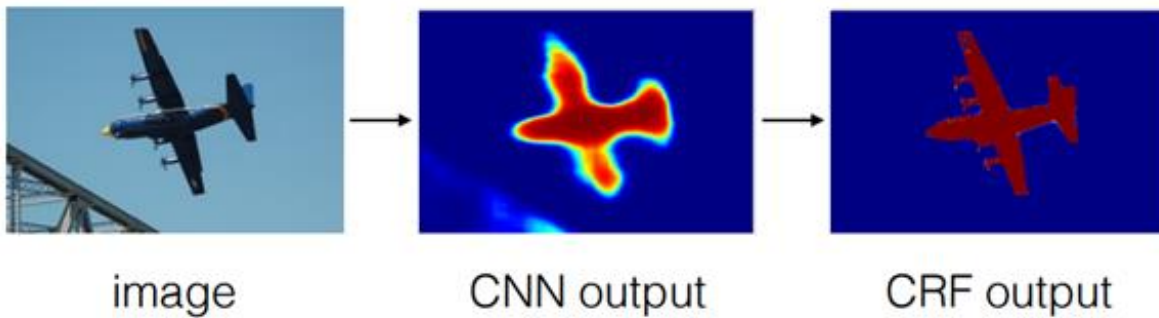
Related Works

- High-quality Segmentation
 - Input Image와 동일한 해상도의 Segmentation을 얻는 과정에서, Feature Map의 해상도를 유지하면 좋겠으나 Memory 문제로 해상도 유지불가
 - 논문에서는 High-quality segmentation을 위한 접근으로 크게 두가지를 소개
 - 1. Prediction Mask Quality 향상, 2. Post Processing(mask prediction 후 refinement)

- Post Processing(mask prediction 후 refinement)

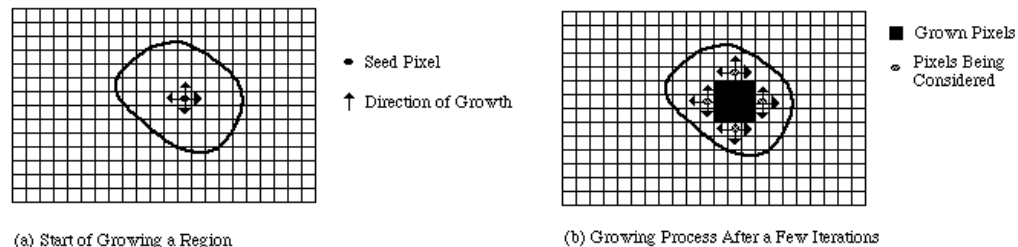
Fully-Connected CRF(Conditional Random Field), DenseCRF라고도 불림.

- DeepLab에서 제안됨 (Semantic Segmentation)
- Model의 결과와 RGB 영상(입력영상)을 조합하여 CRF 계산
- Pixel들간의 관계 (RGB값)을 분석하여 Mask를 Refinement



Region Growing

- Seed Pixel을 선택 후 해당 Pixel과 Intensity가 유사한 것들을 선정해 나가는 과정

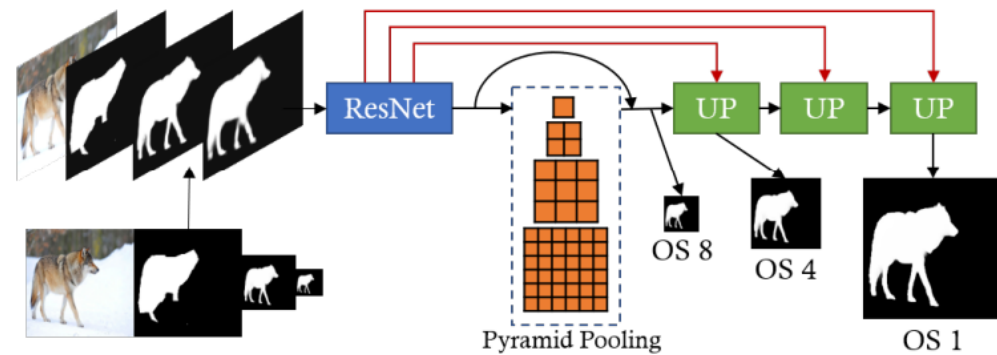
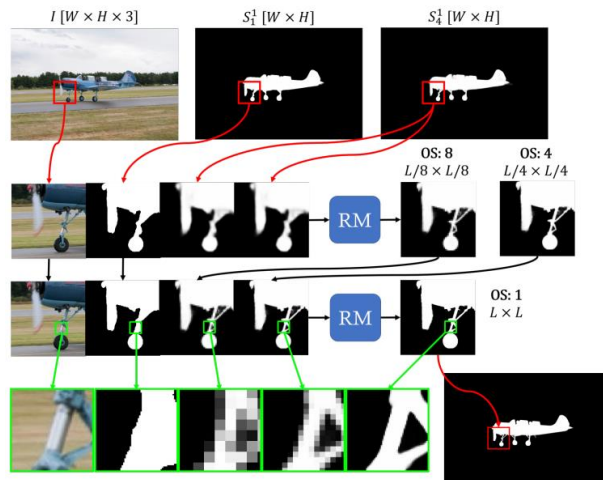


Related Works

- High-quality Segmentation
 - Input Image와 동일한 해상도의 Segmentation을 얻는 과정에서, Feature Map의 해상도를 유지하면 좋겠으나 Memory 문제로 해상도 유지불가
 - 논문에서는 High-quality segmentation을 위한 접근으로 크게 두가지를 소개
 - 1. Prediction Mask Quality 향상, 2. Post Processing(mask prediction 후 refinement)
- Post Iterative refinement

CascadePSP: Toward Class-Agnostic and Very High-Resolution Segmentation via Global and Local Refinement

- 본 논문에서 비교하기 위한 Refinement Method로서 활용한 네트워크 중 하나
- Multi Scale의 Feature Map을 Iterative하게 Refinement 진행하여 Mask Quality를 증가시킴



Related Works

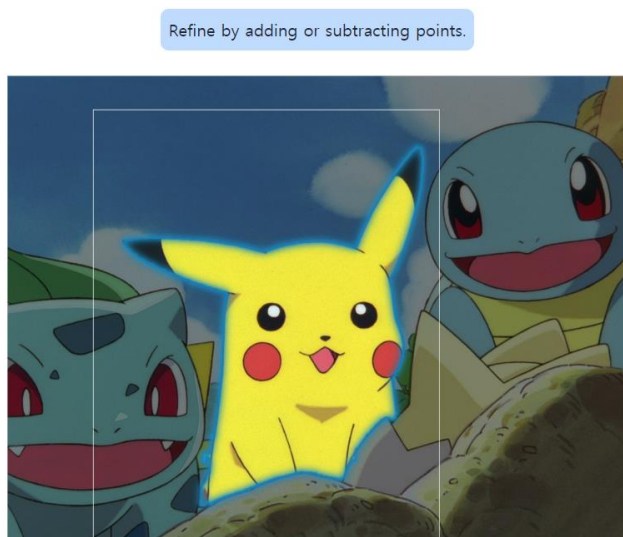
- High-quality Segmentation
 - Input Image와 동일한 해상도의 Segmentation을 얻는 과정에서, Feature Map의 해상도를 유지하면 좋겠으나 Memory 문제로 해상도 유지불가
 - 논문에서는 High-quality segmentation을 위한 접근으로 크게 두가지를 소개
 - 1. Prediction Mask Quality 향상, 2. Post Processing(mask prediction 후 refinement)

본 논문에서는, SAM의 Zero-shot Segmentation 성능 보존을 위해 기존 방법들과 달리 거친 추정 마스크를 다시 가공하거나 입력 이미지를 따로 Refinement Network에 입력하는 방법을 사용하지 않음. Encoder와 Decode를 재사용하여 HQ Mask를 바로 획득

Related Works

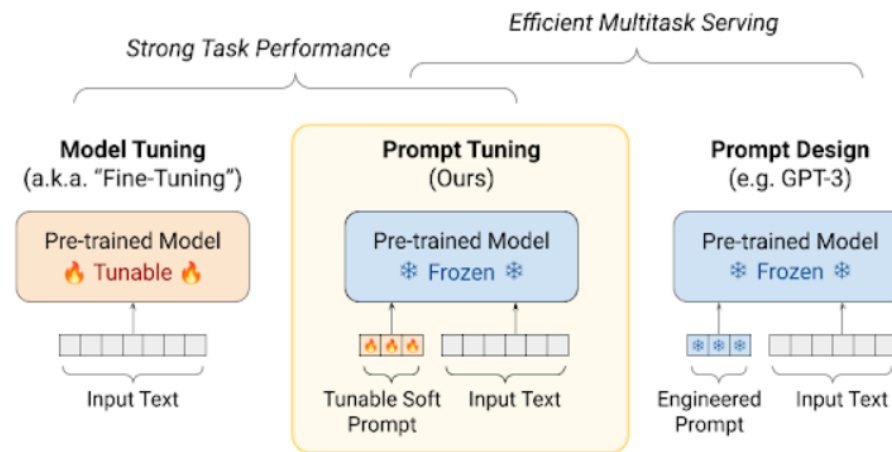
- Prompt Tuning

- **Foundation model**은 같은 이미지(Input)라도 prompt를 주는 것에 따라 **다른 결과**를 얻을 수 있다.
- 주어진 Prompt에 대해 원하는 Output이 나오게 하도록, Pretrained (Foundation) model은 Freeze **시키고 Learnable Parameter를 이용해 Prompt를 Tuning (학습) 하는 것**
- Fine-tuning에 비해 학습하는 시간과 메모리를 절약.



높은 zero-shot transfer 능력을 가지고 있어 inference 시에 자유도가 굉장히 높아진다

Zero-shot transfer: 한번도 보지 못한 class에 대해 task 수행이 가능.

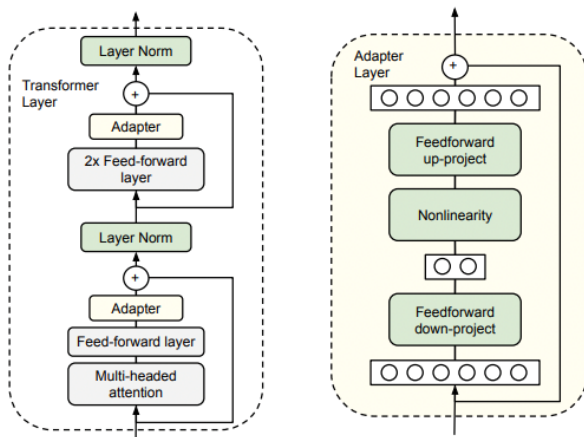


Related Works

- Prompt Based Learning

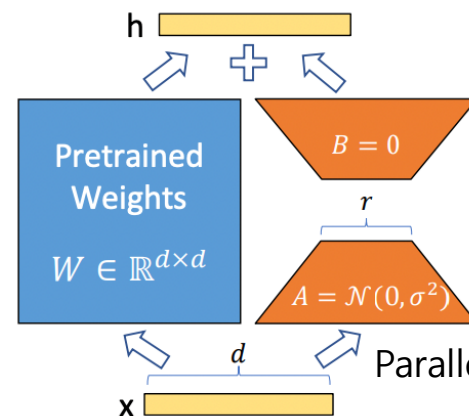
- 이전의 모델의 파라미터를 건드리지 않고 부분만 학습하여 튜닝 (partial tuning)

Adapter module : sequential 한 layer 추가



- LLM 모델의 Attention layer 사이에 학습파라미터를 조금만 추가하여 학습

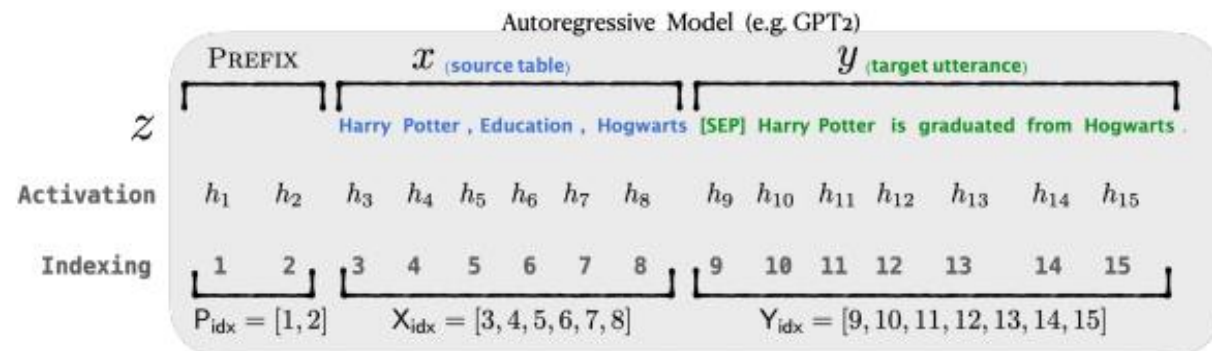
LoRA: Low-rank adaptation of large language models



- Task specific prompt를 만드는 것이 중요
- 특정 prompt를 더 잘 학습하는 Linear layer 추가 적용

Parallel한 layer 추가

Prefix-Tuning: Optimizing Continuous Prompts for Generation



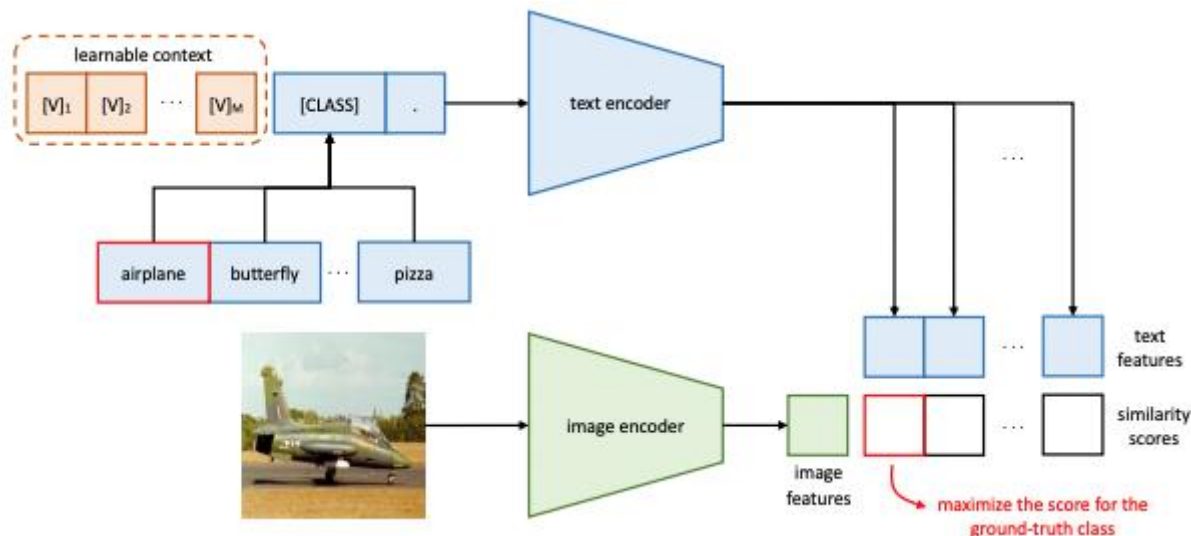
- Learnable parameter를 문장앞에 넣어 continuous한 prompt를 추가

Related Works

- Prompt Based Learning

- 이전의 모델의 파라미터를 건드리지 않고 부분만 학습하여 튜닝 (partial tuning)

- Learning to Prompt for Vision-Language Models (Context Token)



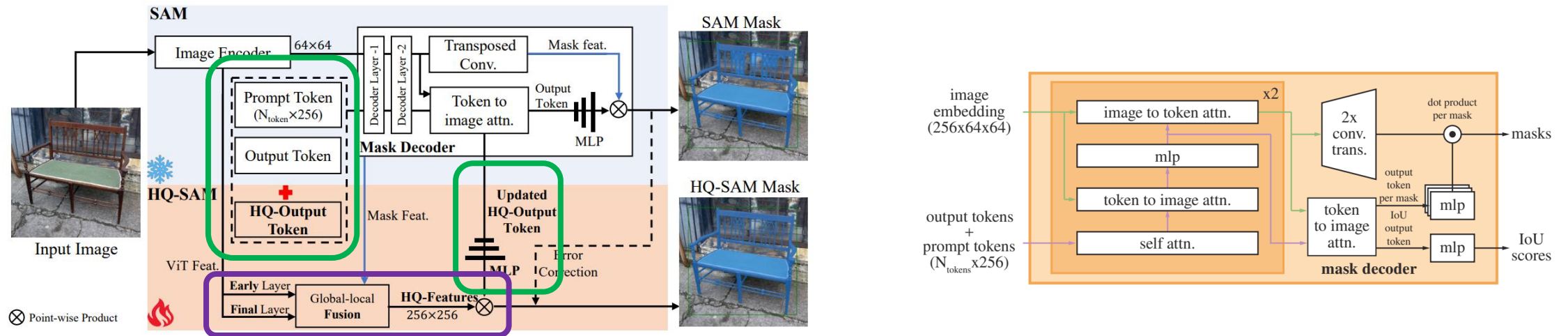
- NLP에서 continuous prompt learning(prefix-tuning)에 영향을 받아
 - Context Optimization 을 제안한 논문
- > context 를 벡터화하여 이 자체로 모델에 학습

Caltech101	Prompt	Accuracy
	a [CLASS].	82.68
	a photo of [CLASS].	80.81
	a photo of a [CLASS].	86.29
	$[V]_1 [V]_2 \dots [V]_M [CLASS]$.	91.83

결과적으로 downstream task에 좋은 성능을 보임

HQ-SAM Model

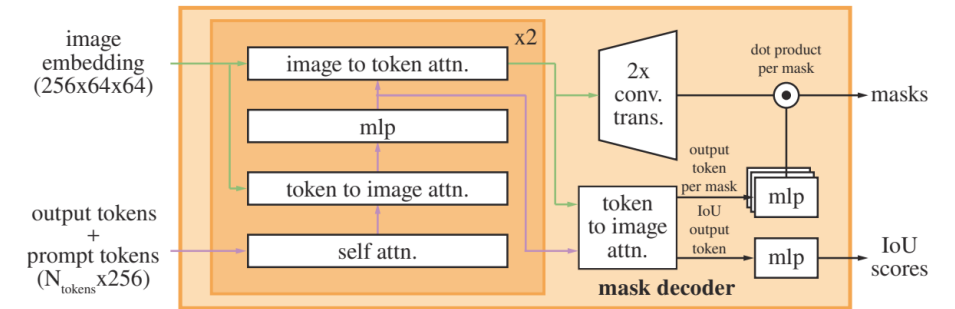
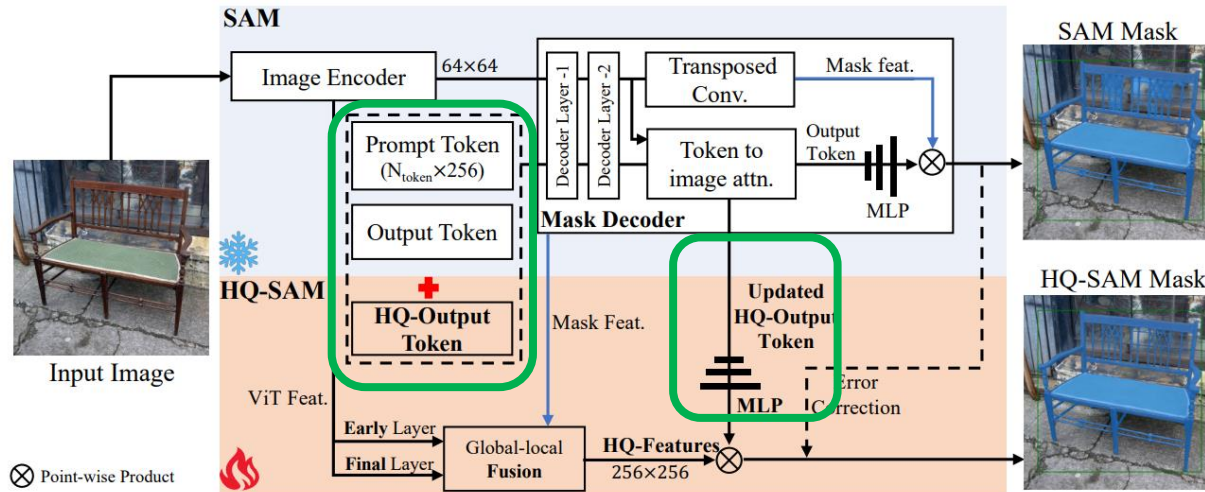
- HQ-SAM에서 추가된 사항



- HQ-Output Token: Mask Decoder에 넣는 또다른 Output token 추가
- Global-local Fusion: Global(전체 형태)과 local feature(texture, 세부형태)에 대한 이해

HQ-SAM Model

- HQ-SAM 토큰



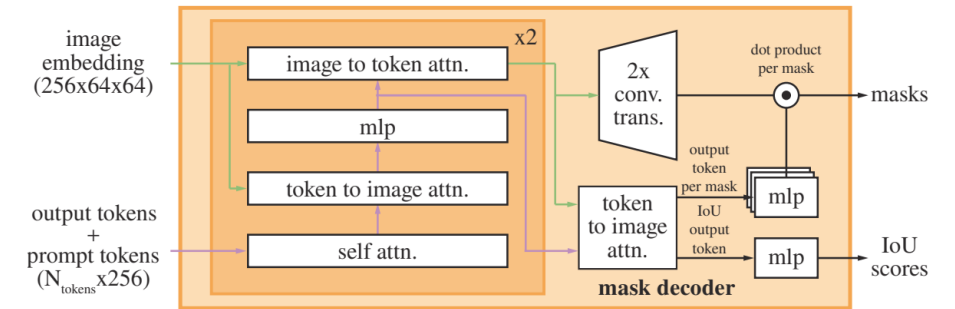
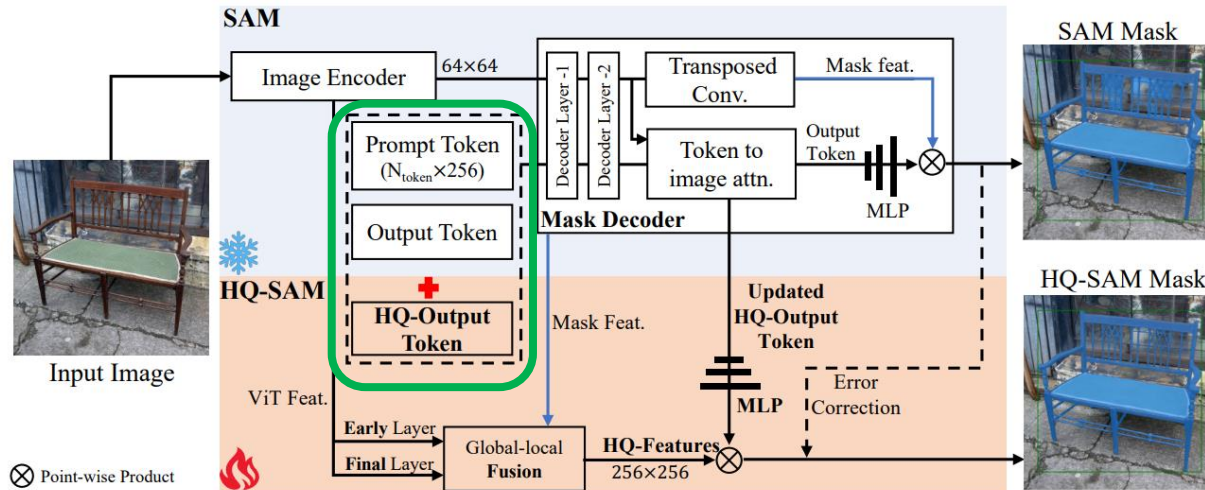
Output Token vs HQ-Output Token vs IoU Token

(4 x 256) vs (1 x 256) vs (1 x 256)

- HQ output token의 Output token에 대한 차이점은 토큰의 개수를 제외하고는 없다. (4개 vs 1개)
- Output token과 마찬가지로 Decoder layer (self attention + token to image + image to token) 통과하면서 image와 prompt 사이의 관계를 token에 담겨지도록 학습.
- SAM에 parameter를 조금 끼워넣은 거라서 시간과 데이터 측면에서 효과적.
- Token하고 mlp layer(Updated HQ-Output Tokens 부분)는 특정 dataset에 overfitting 되지 않는다. 이미 SAM은 거대한 dataset을 학습한 후라, catastrophic forgetting 없이 SAM의 새로운 이미지에 대한 zero shot segmentation 능력은 유지하면서 segmentation 성능을 올릴 수 있음.

HQ-SAM Model

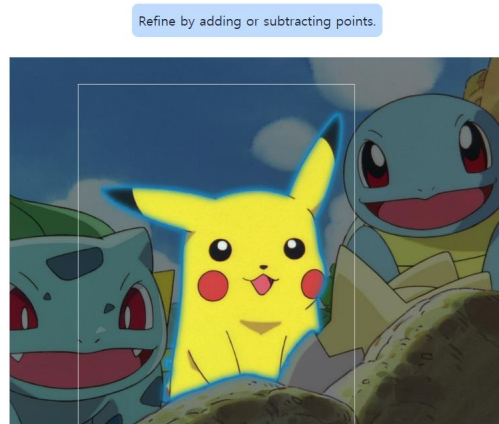
- HQ-SAM 토큰



Output Token vs HQ-Output Token vs IoU Token

(4 x 256) vs (1 x 256) vs (1 x 256)

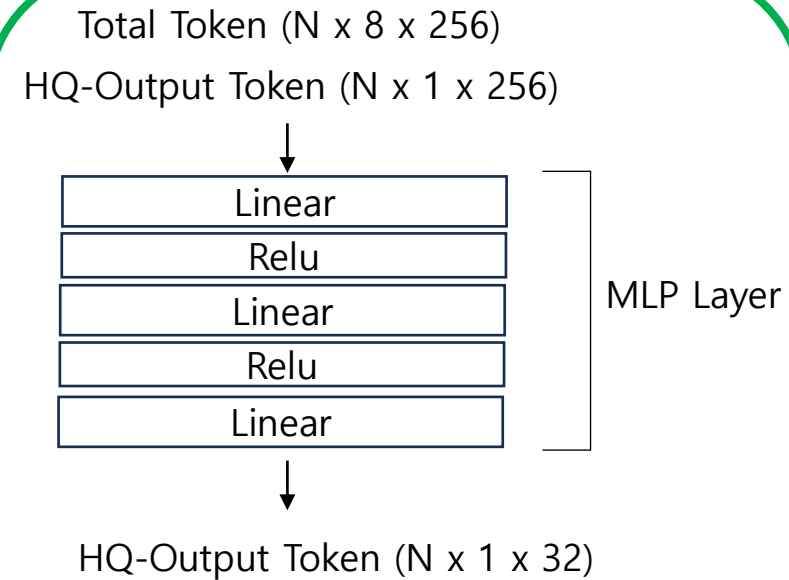
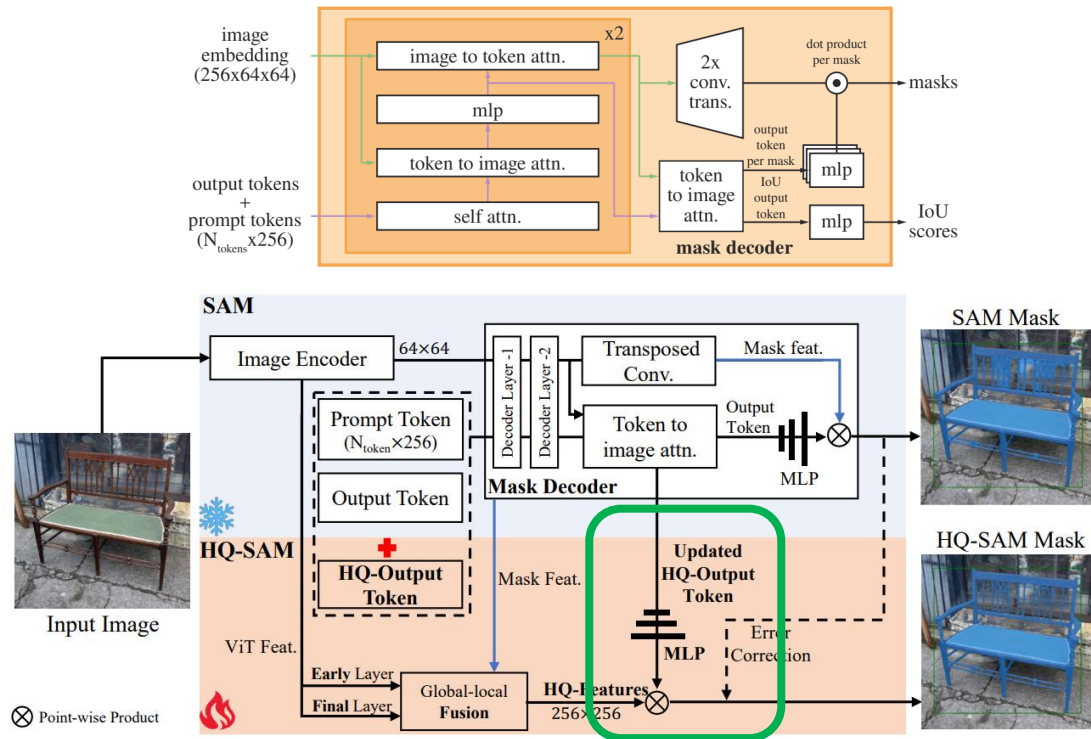
- N개의 Box Prompt가 주어졌을 때



- Box Prompt Token ($N \times 2 \times 256$)
- Output Token ($N \times 4 \times 256$)-> 보통은 mask predict시 3개의 mask를 쓰지만 multiple prompt가 들어왔을 때, 별도로 하나의 token만 쓴다.
- IoU Token ($N \times 1 \times 256$)
- HQ-Output Token ($N \times 1 \times 256$)
- Total Token ($N \times 8(2+4+1+1) \times 256$)

HQ-SAM Model

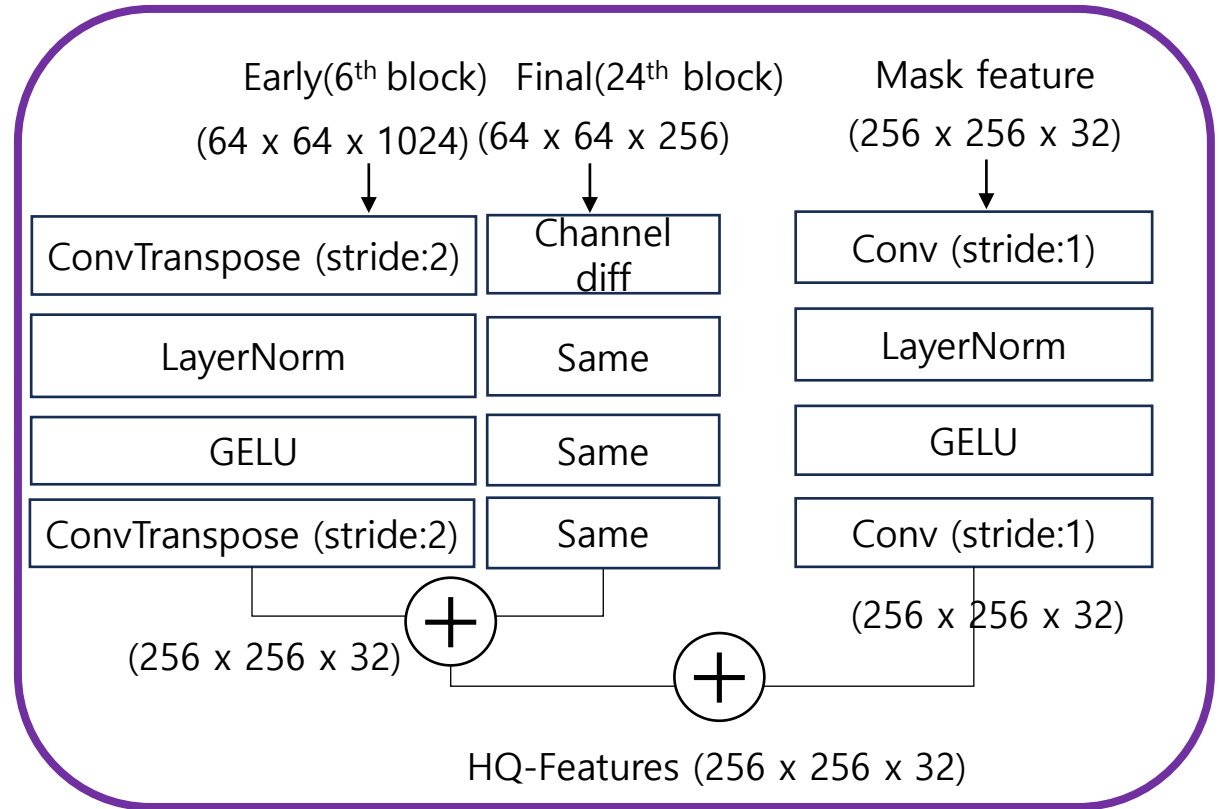
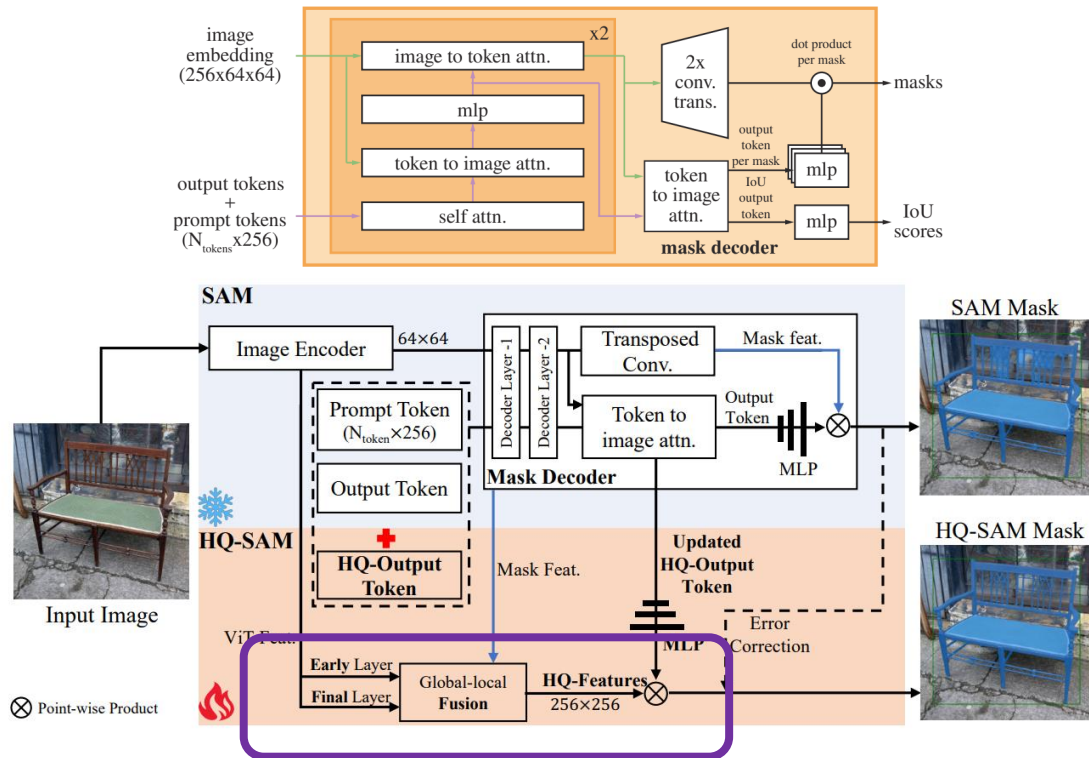
- HQ-SAM 토큰



- SAM에서의 Output token과 동일하게 Mask Decoder Output중에 HQ-Output만 골라서 MLP (Linear Layer 3개)에 넣어줌.
- SAM의 Output token에서의 마스크 에러를 수정하기 위해 HQ-output token을 도입하고 MLP를 학습시킴.

HQ-SAM Model

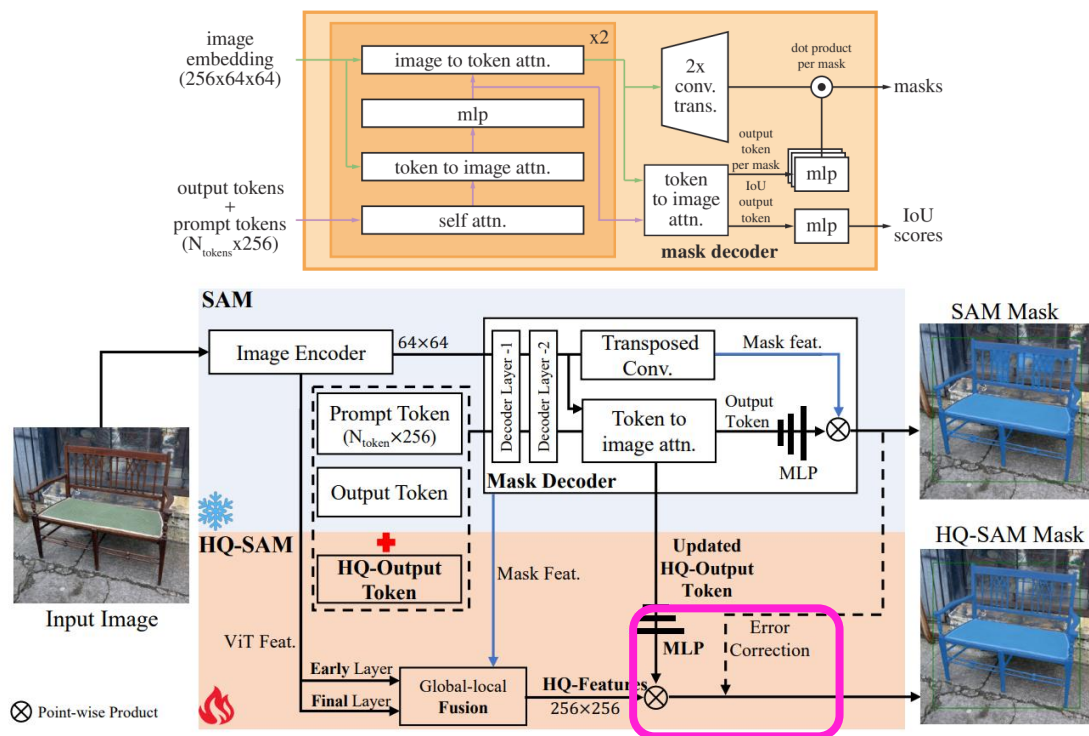
- Global-local Fusion



- 전체 global feature(전체적인 모양이나 구도)과 local feature(boundary 모양) 둘 다 중요.
- CNN과 비슷하게 early layer는 local feature (boundary), final layer는 global feature를 주로 추출함.
- Mask feature는 Image embedding가 Mask decode를 통해 나온 Output이며, 명확하게 마스크 모양 정보를 갖고 있다.

HQ-SAM Model

- 최종적인 Output



HQ-Output Token ($N \times 1 \times 32$)

Dot Product $\otimes$ $\rightarrow$ HQ Mask ($N \times 256 \times 256 \times 1$)

HQ-Features ($N \times 256 \times 256 \times 32$)
(Early+Final+Mask feature)

Total Masks ($N \times 256 \times 256 \times 5$) $\rightarrow$ 예측되는 IoU Score에 따라 4개의 마스크 중 하나 선정

Output Token ($N \times 4 \times 32$)

Dot Product $\otimes$ $\rightarrow$ Masks ($N \times 256 \times 256 \times 4$)

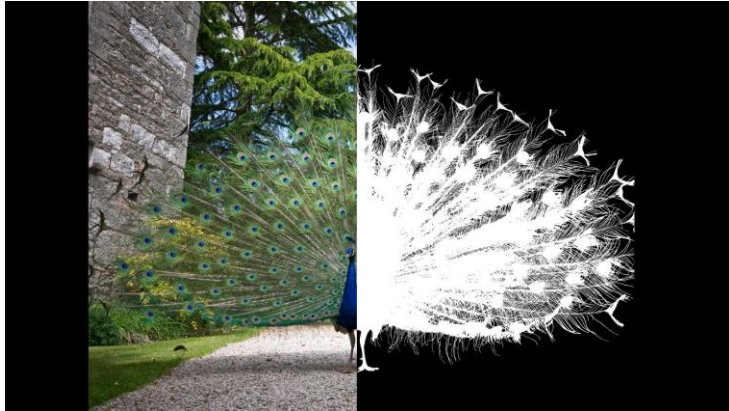
Mask feature ($N \times 256 \times 256 \times 32$)

- HQ feature는 **image embedding의 feature**를 갖고 있고, HQ-Output Token은 **attention에 대한 channel상의 정보**를 갖고 있어 둘이 dot product하면, 최종적인 loss를 최소로 하는 mask정보를 구할 수 있다.
- 원래 SAM masks ($256 \times 256 \times 3$)와 HQ-mask ($256 \times 256 \times 1$)와 IoU prediction이 **더 높은 mask**를 골라 upsample해서 (1024×1024)의 mask 최종적인 output을 만들어낸다.

Methods

- Training Data Construction

DIS



ThinObject



Figure 4: Example images from our ThinObject-5K dataset. Please zoom-in for more details.

SAM의 dataset SA-1B (10억개 이미지) 대신 HQSeg-44K (44320개의 이미지)를 추가로 학습.

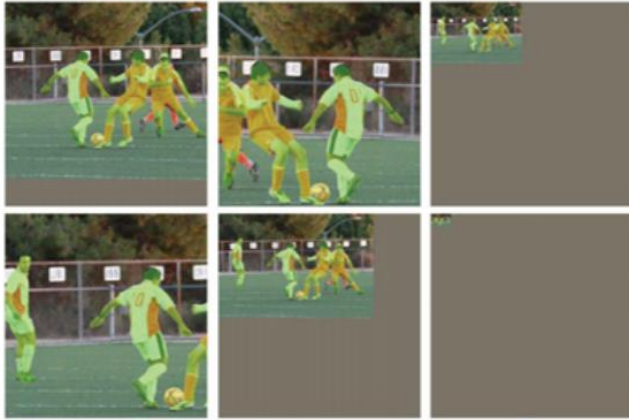
6개의 이미 존재하는 Dataset들을 활용(DIS, ThinObject, FSS, ECSSD, MSRA, DUT-OMRON)

Annotation 자동생성된거랑 아닌거랑 섞여있는 SAM 모델과 다르게 여기서는 굉장히 정확하게 annotation된 걸 씬.

General한 segmentation을 위해 1000개 이상의 semantic classes를 갖음.

Methods

- HQ-SAM Training



(b) Large Scale Jittering (LSJ)

- 최종적으로 HQ-SAM에서 학습되는 것은 3-layer MLP, 그리고 3개의 conv로 이루어진 HQ-Features fusion 뿐이다.
- Bounding box(+random gaussian noise), 랜덤 샘플 포인트, 좀 러프한 마스크 input을 만들어서 학습 시킴 + Large Scale Jittering(LSJ) 적용해서 물체 scale에 대해 generalization함.
- 마스크 input은 GT mask에 Gaussian noise를 경계 지역에 추가해서 성능이 떨어지는 마스크를 만들어냄.
- Lr 0.001이고 12epoch 10 epoch후에 learning rate 떨어뜨림
- 8 RTX 3090 GPU를 써서 32 batch size로 4시간 학습시킴.

Methods

- HQ-SAM Inference

Table 1: Training and inference comparison between ViT-L [11] based SAM and HQ-SAM. HQ-SAM brings negligible extra computation burden to SAM, with *less than 0.5% increase* in model parameters and reaching 96% of its original speed. SAM-L is trained on 128 A100 GPUs for 180k iterations. Based on SAM-L, we only need to train our HQ-SAM on 8 RTX3090 GPUs for 4 hours.

Method	Learnable Params (M)	Training			Inference	
		# GPU	Batch Size	Time (h)	FPS	Mem.
SAM [21]	1191	128	128	N/A	5.0	7.6G
HQ-SAM	5.1	8	32	4	4.8	7.6G

- Inference시 FPS와 Memory 차이가 크지 않음.

Experiments

- Experiment Setup

- **Datasets:** Extremely fine-grained dataset과 general한 데이터셋에 대해서 성능 평가함

- Extremely fine-grained segmentation dataset: DIS (valid set), ThinObject-5K (test set), COIFT, HR-SOD
- Popular & Challenging Benchmarks dataset: COCO, UVO, LVIS, HO-YTVIS, BIG

- **Evaluation Metrics:** 일반적인 **mIoU**, **AP**와 더불어서 다른 metric도 사용함

- Boundary IoU, Boundary AP
- AP_B^{strict} (Strict Boundary AP)의 경우 dilation ratio를 0.02에서 0.01로 조절한 조금 더 strict한 기준이다.

Boundary IoU

$$\text{Boundary IoU} = \frac{\text{Area} \left(|(G_d \cap G) \cap (P_d \cap P)| \right)}{\text{Area} \left(|(G_d \cap G) \cup (P_d \cap P)| \right)}$$

Given ground truth mask G and prediction mask P , Boundary IoU first computes the set of the original masks' pixels that are within distance d from each contour, and then computes the intersection-over-union of these two sets:

$$\text{Boundary IoU}(G, P) = \frac{|(G_d \cap G) \cap (P_d \cap P)|}{|(G_d \cap G) \cup (P_d \cap P)|},$$

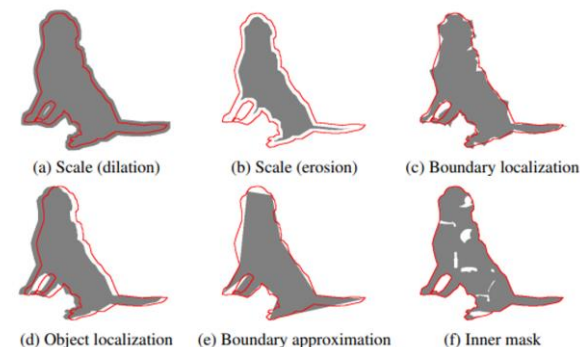


Figure 2: **Examples of error types** generated from a single ground truth mask. The red contour represents the ground truth contour. *Scale errors*: (a) dilation and (b) erosion of the mask. *Boundary localization error*: (c) adding random Gaussian noise to each vertex in polygons. *Object localization error*: (d) shifting masks. *Boundary approximation error*: (e) simplifying polygons. *Inner mask error*: (f) adding holes to masks.

Experiments

- Ablation Study

Effect of the High-Quality Output Token.

Table 2: Ablation study of the HQ-Output Token on four extremely fine-grained segmentation datasets. We adopt the boxes converted from their GT masks as the box prompt input. By default, we train the predicted mask of HQ Output-Token by computing full GT mask loss.

Model	DIS [34]		COIFT [29]		HRSOD [50]		ThinObject [29]		Average	
	mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU
SAM (baseline)	62.0	52.8	92.1	86.5	90.2	83.1	73.6	61.8	79.5	71.1
<i>Using SAM's mask decoder feature:</i>										
SAM + Context Token [54]	71.5	62.2	93.0	87.7	91.8	85.0	84.5	73.1	85.2	77.0
SAM + HQ-Output Token ($\times$ Output Token)	75.1	65.8	93.9	88.9	93.0	86.1	86.1	74.6	87.0	78.9
SAM + HQ-Output Token (Boundary Loss)	75.2	66.4	94.0	88.9	92.1	85.7	87.3	76.0	87.2	79.3
SAM + HQ-Output Token	75.3	66.0	94.2	89.2	93.0	86.1	86.8	75.4	87.3	79.2
<i>Using Our HQ-Feature:</i>										
SAM + HQ-Output Token (+ Context Token)	78.5	70.4	94.6	89.6	93.6	87.0	88.9	79.3	88.9	81.6
SAM + HQ-Output Token	78.6	70.4	94.8	90.1	93.6	86.9	89.5	79.9	89.1	81.8

- HQ-output token의 성능**을 평가하기 위해서 현존하는 prompt/token learning 방법인 Context Token과 비교를 진행.
- Context Token 방법을 도입한 것보다 **HQ-Output Token**을 활용한 방법들이 더 높은 성능을 보임.
- HQ-Output Token을 활용할 때 original SAM의 feature mask에 dot product를 하는 것과, boundary region 안쪽에 대한 mask의 loss를 계산하여 학습하는 것도 시도해 보았는데 제안한 방법보다는 약간 낮은 성능을 보임.

Experiments

- Ablation Study

Ablation on the Global-local Fusion for HQ-Features.

Table 3: Ablation study on the HQ-Features sources. Early-layer denotes the feature after the first global attention block of the ViT encoder, while final-layer denotes the output of the last ViT block. Four HQ datasets denote DIS (val) [34], ThinObject-5K (test) [29], COIFT [29] and HR-SOD [50].

Model	Fusion conv	Decoder Mask feature	ViT Encoder		Four HQ datasets	
			Final-layer	Early-layer	mIoU	mBIoU
SAM [21]		✓			79.5	71.1
HQ-SAM (Ours)		✓			87.3	79.2
	✓	✓			87.8	80.1
	✓		✓		15.1	9.0
	✓	✓	✓		88.6	81.3
	✓	✓	✓	✓	89.1	81.8

- SAM 디코더의 feature를 바로 사용하는 것보다 HQ-Feature를 사용했을 때 mBIoU가 2.6정도 상승.
- Fusion conv을 도입하고, Final layer와 Early layer를 모두 사용하는 것이 가장 좋은 성능을 보인다.

Experiments

- Ablation Study

Comparison to SAM finetuning or post-refinement.

Table 4: Comparison with model finetuning or extra post-refinement [6]. For the COCO dataset, we use a SOTA detector FocalNet-DINO [51] trained on the COCO dataset as our box prompt generator.

Model	Four HQ datasets		COCO				
	mIoU	mBIOU	AP_B	AP	AP_L	AP_M	AP_S
SAM (baseline)	79.5	71.1	33.3	48.5	63.9	53.1	34.1
Add Context Token [54]	85.2	77.0	31.9	47.2	65.1	51.2	31.9
Extra Post-refinement [6]	80.9	74.6	2.8	13.4	43.4	9.4	0.0
Finetune SAM's decoder	87.6	79.5	9.0	19.5	45.2	15.8	4.7
Finetune SAM's output token	87.6	79.7	33.7	48.7	66.0	52.3	33.6
HQ-SAM (Ours)	89.1	81.8	34.4	49.5	66.2	53.8	33.9

- 무거운 **post-refinement** 네트워크를 추가했을 때 Four HQ dataset에 대해서는 높은 성능을 보이나, **COCO**에 대해서는 아주 낮은 성능을 보이고 있으므로 **overfitting**이 되었다고 할 수 있다.
- SAM의 decoder를 직접적으로 finetuning했을 때도 비슷한 결과를 볼 수 있었다.
- SAM의 output token만을 finetuning했을 때는 overfitting 현상은 덜했으나 HQ-SAM보다는 조금 낮은 성능을 보이고 있다.

Experiments

- Ablation Study

Accuracy analysis at different BIoU thresholds.

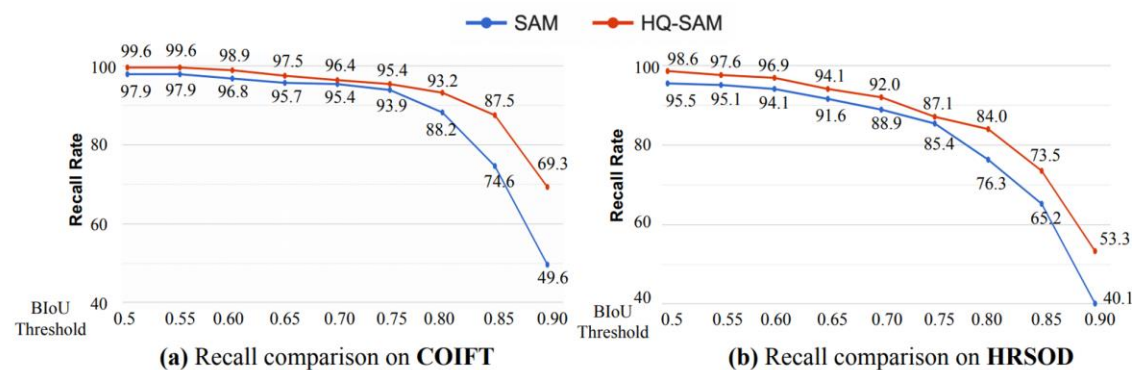


Figure 3: Recall rate comparison between COIFT [29] and HRSOD [50] under the zero-shot protocol, using BIoU thresholds ranging from loose to strict. The performance gap between SAM and our HQ-SAM increases significantly when we vary from a loose BIoU threshold of 0.5 to a very strict threshold of 0.9, showing the advantage of HQ-SAM in predicting very accurate segmentation masks.

- Strict한 BIoU threshold값을 설정할수록 HQ-SAM의 더 좋은 Recall Rate를 보인다.
- Boundary 지역을 segmentation하는 데에 HQ-SAM의 성능이 좋다.

Experiments

- Zero-shot Comparison with SAM

Zero-shot Open-world Segmentation

Table 5: Zero-shot open-world instance segmentation results comparison on UVO [41]. We use FocalNet-DINO [51] trained on the COCO dataset as our box prompt generator. $*^{strict}$ denotes the boundary region with a tighter threshold.

Model	AP_B^{strict}	AP_{B75}^{strict}	AP_{B50}^{strict}	AP_B	AP_{B75}	AP_{B50}	AP
SAM	8.6	3.7	25.6	17.3	14.4	37.7	29.7
HQ-SAM	9.9	5.0	28.2	18.5	16.3	38.6	30.1

- Diverse and Dense objects mask annotation을 가지고 있는 UVO 데이터셋에 대해서 성능을 평가했다.
- 같은 object detector를 사용하여 bbox를 prompt로 주었을 때, HQ-SAM이 더 좋은 성능을 보였다.

Experiments

- Zero-shot Comparison with SAM

Zero-shot Visual Results Comparison

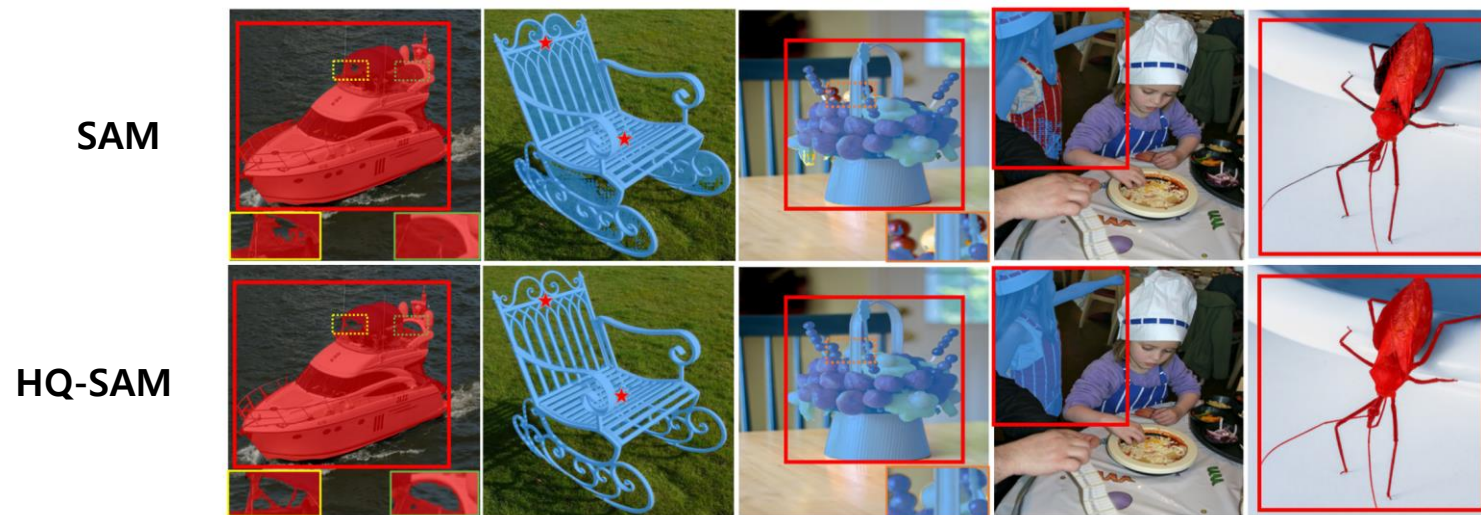


Figure 4: Visual results comparison between SAM (top row) vs. HQ-SAM (bottom row) in a *zero-shot transfer setting*, given the same red box or point prompt. HQ-SAM produces significantly more detailed-preserving results and also addresses the mask errors with broken holes.

large portion error



- HQ-SAM이 broken hole이나 large portion error같은 경우를 잘 보완해주었다.

Experiments

- (Appendix) Effect of HQSeg-44K

Table 14: Data composition of our constructed HQ-Seg-44K.

Dataset	DIS [34]	Thin-Object 5k [29]	FSS [26]	DUTS [45]	ECSSD [37]	MSRA-10K [8]	Total
Image Num.	3000	4748	10000	15572	1000	10000	44320

Table 15: Comparison of the training dataset. For the COCO dataset using ViT-L-based SAM, we use a SOTA detector FocalNet-DINO [51] trained on the COCO dataset as our box prompt generator.

Model	Dataset	DIS		COIFT		HRSOD		ThinObject		Average	
		mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU	mIoU	mBIOU
SAM	SA-1B	62.0	52.8	92.1	86.5	90.2	83.1	73.6	61.8	79.5	71.1
HQ-SAM	+ SA-1B-44K	60.4	51.7	91.1	86.1	88.4	80.9	73.1	61.8	78.3	70.1
HQ-SAM	+ HQ-Seg-44K (Ours)	78.6	70.4	94.8	90.1	93.6	86.9	89.5	79.9	89.1	81.8

- HQSeg-44K에 사용된 데이터셋과 training data를 서로 달리 하였을 때의 결과를 보여준다.
- 기존 SAM의 경우 SA-1B에 대해서 학습을 했고, HQ-SAM은 이렇게 학습된 weight를 freeze한 채 HQ Token만 추가적으로 학습하게 된다.
- SA-1B에서 44K개를 랜덤으로 골라서 학습시켰을 때와 HQ-Seg-44K 데이터셋으로 학습시켰을 때를 비교했을 때 **HQ-Seg에**서 훨씬 더 높은 성능을 보이고 있다.

Review & Discussion (개인의견)

- Global Local Fusion의 영향보다는 **HQ token(새 Output token)**을 이용해 좋은 데이터셋(HQSeg44K)를 잘 훈련시킨 영향이 더 커보인다.

Model	Dataset	DIS		COIFT		HRSOD		ThinObject		Average	
		mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU
SAM	SA-1B	62.0	52.8	92.1	86.5	90.2	83.1	73.6	61.8	79.5	71.1
HQ-SAM	+ SA-1B-44K	60.4	51.7	91.1	86.1	88.4	80.9	73.1	61.8	78.3	70.1
HQ-SAM	+ HQ-Seg-44K (Ours)	78.6	70.4	94.8	90.1	93.6	86.9	89.5	79.9	89.1	81.8

Model	DIS [34]		COIFT [29]		HRSOD [50]		ThinObject [29]		Average	
	mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU	mIoU	mBIoU
SAM (baseline)	62.0	52.8	92.1	86.5	90.2	83.1	73.6	61.8	79.5	71.1
Using SAM's mask decoder feature:										
SAM + Context Token [54]	71.5	62.2	93.0	87.7	91.8	85.0	84.5	73.1	85.2	77.0
SAM + HQ-Output Token ($\times$ Output Token)	75.1	65.8	93.9	88.9	93.0	86.1	86.1	74.6	87.0	78.9
SAM + HQ-Output Token (Boundary Loss)	75.2	66.4	94.0	88.9	92.1	85.7	87.3	76.0	87.2	79.3
SAM + HQ-Output Token	75.3	66.0	94.2	89.2	93.0	86.1	86.8	75.4	87.3	79.2
Using Our HQ-Feature:										
SAM + HQ-Output Token (+ Context Token)	78.5	70.4	94.6	89.6	93.6	87.0	88.9	79.3	88.9	81.6
SAM + HQ-Output Token	78.6	70.4	94.8	90.1	93.6	86.9	89.5	79.9	89.1	81.8

Model	Four HQ datasets		COCO				
	mIoU	mBIoU	AP _B	AP	AP _L	AP _M	AP _S
SAM (baseline)	79.5	71.1	33.3	48.5	63.9	53.1	34.1
Add Context Token [54]	85.2	77.0	31.9	47.2	65.1	51.2	31.9
Extra Post-refinement [6]	80.9	74.6	2.8	13.4	43.4	9.4	0.0
Finetune SAM's decoder	87.6	79.5	9.0	19.5	45.2	15.8	4.7
Finetune SAM's output token	87.6	79.7	33.7	48.7	66.0	52.3	33.6
HQ-SAM (Ours)	89.1	81.8	34.4	49.5	66.2	53.8	33.9

- Global Local fusion을 이용하지 않으면, HQ-Output token과 Output-token을 학습시키는 것은 동일하다. (토큰 개수의 차이)
- SAM에서 Output Token을 fine-tuning하는 방식이 SAM의 높은 **zero shot transfer** 성능은 유지시키면서 **segmentation task**는 향상 시킨다.
- 무엇이든 Segmentation 잘 하는 SAM (zero-shot transfer)을 따로 HQSeg44K라는 얇은 선이 잘 따진 데이터셋으로 효율적으로(HQ Token을 이용) 훈련시켜서 어떤 object든 얇은 선을 잘 딸 수 있게 만든 모델 (HQ-SAM) -> 앞으로 다른 데이터셋으로 추가 학습시킨 **SAM v2, v3** 등이 출시될 수도.
- 다양한 Segmentation 방식을 내 입맛에 맞게 Tuning하고 싶을 때, Output token을 더 추가해서 학습(HQ-SAM과 동일하게)시킨다.
- 토큰 자체를 Downstream Task를 할 때의 Prompt Tuning 관점으로 본다면, 다른 domain에서의 segmentation이나 다른 기능을 추가할 때마다 이런 Token을 여러 개를 추가하면 되지 않을까? (추정 마스크의 개수와 동일)

Ex) Medical Image (Domain), Depth(thermal) estimation per segmentation instance

Thank you!

Question & Answer